

P1634R0

Naming guidelines for modules

Published Proposal, 2019-06-16

Author:

[Corentin Jabot](#)

Audience:

SG15

Project:

ISO/IEC JTC1/SC22/WG21 14882: Programming Language — C++

Abstract

In which we propose a set of rules and advice to name modules

Table of Contents

1 Introduction

2 What problems are we trying to solve

3 Applicability

4 Proposed guidelines

- 4.1 Prefix module names with an entity and/or a project name to prevent modules from different companies, entities and projects of declaring the same module names.
- 4.2 Use lower-case ASCII module names
- 4.3 Prefer organizing modules hierarchically.

5 Other considerations and future work

- 5.1 Dependency management
- 5.2 Private projects

5.3 Toolability

5.4 Use consistent names

References

Informative References

§ 1. Introduction

Modules names are unique in a C++ program (all modules units identified by the same name refer to the same module). Furthermore, the name of a module is part of the API and ABI. This is different from headers which are usually identified by a path which can be adapted by consumers, and which are never part of the ABI.

And so, renaming a module is a very disruptive change within a project and more importantly, it's a very disruptive breaking change for downstream projects. In other words, modules can virtually never be renamed.

As code is poised to be reused and shared across companies, communities, and projects, the requirement that modules need to be uniquely identified within a project directly translate to the requirement that module names be unique across the universe.

While offering such a guarantee is impossible, the following guidelines strive to reduce the risk of naming conflicts.

§ 2. What problems are we trying to solve

A historical lack of tooling contributed to C++ projects having shallow dependency trees, in which conflicts are not frequent - but do occur (`util.hpp`, `config.h` and other are very frequent header names). In recent years we observe a desire in the community for tools that would allow deeper dependency trees. Dependency trees can grow exponentially and as the tooling and the ecosystem mature, the risk of naming conflict increase. Because modules cannot be renamed without breaking API, naming conflicts would impede the community's ability to provide these tools and interoperable libraries. We came to understand that names declared at global scoped or imported into the global scope by `using namespace` statements hamper our ability both to add names to a library or to update that library in dependent code, the same problem would inevitably occur in the absence of rules designed to minimize the risk of conflict.

We can not predict the success we will have as a community in making it easier to reuse our code but we know modularizing and making our existing libraries dependable "at scale" will be challenging.

In the case of modules naming and mapping, we have a unique opportunity to be proactive, which is much easier than trying to change our codebases in 2, 5, 10 years. And while modules are new and unfamiliar, existing challenges with headers as well as extensive experience by other languages [\[CSHARP\]](#)[\[JAVA\]](#)[\[Python\]](#) can help us design guidelines that will ensure naming conflicts are unlikely to arise as the number of libraries increases [over time](#) [\[past-vs-future\]](#).

§ 3. Applicability

The proposed guidelines are for the benefits of library authors who intend to share code with external entities, either open source or not. Programs (root node in dependency graphs) are still encouraged to follow the following guidelines as naming conflicts can still arise with imported modules, existing code and company policies permitting.

§ 4. Proposed guidelines

- **Prefix module names with an entity and/or a project name to prevent modules from different companies, entities and projects of declaring the same module names.**
- **Use lower-case ASCII module names.**
- **Prefer organizing modules hierarchically**

§ 4.1. Prefix module names with an entity and/or a project name to prevent modules from different companies, entities and projects of declaring the same module names.

A good way to prevent conflict is to prefix all modules with the name of the entity maintaining the project. This scheme has, with variation, proved successful in many other languages faces with the same issues (C#, JavaScript, Go, ...).

As such, `google.abseil` is a better (more unique) module name than `abseil`. The logic is that `google` is probably unique - companies and individuals usually have to maintain the uniqueness of their name or online identities. In fact, this schema has been adopted by numerous code hosting platforms such that most open source projects can be uniquely identified by the name of their repository, itself in the form `org/project`, extending that naming scheme, therefore, would be consistent with other naming considerations projects have to contend with.

Experience with other languages and with C++ namespaces show that it is unnecessary and indeed counter-productive to recommend more than 2 identifiers to uniquely identify a project. As such we do not, for example, recommend that module be prefixed by a team name.

Languages such that Java use reverse domain names to identify packages (which are, for our purpose, comparable to modules)

For example, a module for the boost Asio library would under this scheme be named `org.boost.asio`.

However, this has a number of drawbacks

- Not every entity has a domain name
- Some entities have several domain names
- It is not unusual for domain names to change
- Developers have to remember the TLD of entities which adds unnecessary cognitive load.

It is also unclear, despite the uniqueness of domain names, whether this scheme actually helps in guaranteeing the uniqueness of a name.

There are exceptions to consider: Projects that are developed under an eponymous umbrella, which have gained notoriety over the years. These projects would have existed long before the present proposal and the C++ ecosystem Technical Report, and they are numerous.

Qt is developed by the Qt project, Catch2 is developed under the name Catch2, etc. It makes such in these cases to recommend `qt` or `catch2` as module name prefixes rather than `qtproject.qt` or `catchorg.catch2`.

The proposed scheme intends to describe in a concise way the ownership of a module and does not need to reflect the internal organization of a company. In that sense, and because short names are easier to remember, only the first identifier of a module name should pertain to the general ownership while the second part pertain to the project. Optional subsequent parts are left to the discretion of the maintainers.

- **DO :** `my_company.my_project`
- **DON'T :** `my_company.my_division_my_team.my_project`

§ 4.2. Use lower-case ASCII module names

In a separate document, we will explore a way to deterministically map a module name to the name of the file declaring its header unit. Such mapping restrict module names to be (or be mappable) to characters valid in file names

Not all filesystems are case sensitive, and as such module names which would differ only by the case could not be mapped to a unique filename. Lack of consistency in case insensitive platforms is an ongoing issue with header names and library names - for example, it is not generally possible to cross-compiling code targeting the Windows platform (Win32 and Windows SDK) APIs from a case sensitive system.

Recent discussions and work within SG15 illustrate the maddening difficulty of portably encode non-ASCII file names, and as such module name with non-ASCII file names would not be directly mappable to a unique filename.

While these rules may sound restrictive, they ensure consistency across various levels of encapsulation (files, modules, namespace) in a way that is portable across platforms and systems. While it may be tempting to support Unicode and case sensitivity it needs to be done in a way that is mindful of portability and security, in a way that permits a future proposal and tools do devise a deterministic and robust name to file mapping.

Because it is possible always possible to relax rules (but never to add constraints at a later date), we think a conservative approach is necessary until we get a better sense of the cost of allowing non-ASCII characters [\[google-style\]](#), in a way that would be consistent with the rest of the language and consistent across implementations neither of which can be done in the C++20 time frame.

§ 4.3. Prefer organizing modules hierarchically.

For example, if both modules `boost.asio` and `boost.asio.executors` exist as part of the public API of `boost.asio`, `boost.asio` should reexport `boost.asio.executor`.

Dots in module names do not introduce a hierarchy. Yet, because of a long history of modules, namespacing and packaging system in existing languages, there is some expectation that nested modules will be reexported.

Doing so ensures consistency in how modules are organized which makes consuming them easier.

This is again encouraging existing practices, ie it is currently expected that `<boost/asio.hpp>` would include `<boost/asio/executor.hpp>`

§ 5. Other considerations and future work

§ 5.1. Dependency management

While modules are not a distribution mechanism, the proposed rules are designed with dependency management in mind, such that a large number of C++ libraries could be consumed within the same project without risk of conflict. They also make it possible to use the same name to identify both the top-level module and the package/library/dependency of which it is part. This, in turn, can be used by smart dependency manager offering automatic dependencies scanning. In this case

§ 5.2. Private projects

The proposed guidelines are intended to ease the reuse of C++ libraries Code that isn't intended to be share does have to ascribe to any rules but still needs to be designed in such a way that it can depend on other libraries and modules.

§ 5.3. Toolability

While outside the purview of this document, the uniqueness of module name as well as the proposed rules could be enforced by a module name registry or a package index.

§ 5.4. Use consistent names

Modules add yet another named layer of encapsulation. But they are not themselves a name scoping mechanism. And so there is a growing number of entities that need to be named: package, libraries, modules, namespace.

It might, therefore, be useful to encourage the following practices, notably because modules do not prevent symbol name to conflict, especially in the global namespace.

- Exported top-level namespaces should have a name identic to the project name used as part of the name of the module(s) from which it is exported [\[google-style\]](#).
- Do not export multiple top-level namespaces
- Do not export entities in the global namespace outside of the global module fragment.

§ References

§ Informative References

[CSHARP]

Names of Namespaces. URL: <https://docs.microsoft.com/en-us/dotnet/standard/design-guidelines/names-of-namespaces>

[GOOGLE-STYLE]

Google C++ Style Guide. URL:
https://google.github.io/styleguide/cppguide.html#Namespace_Names

[JAVA]

Naming a Package. URL: <https://docs.oracle.com/javase/tutorial/java/package/namingpkgs.html>

[PAST-VS-FUTURE]

Titus Winters. Pacific++ 2018: C++ Past vs. Future. URL: <https://www.youtube.com/watch?v=IY8tHh2LSX4>

[Python]

Naming conventions and recipes related to packaging. URL:
<https://www.python.org/dev/peps/pep-0423/>